بسم الله الرحمن الرحیم

Data and Network Security

SESSION 07

# Control-Flow Integrity and Hardware Defenses



**Solmaz Salimi**
**Sharif University of Technology**

s.salimi@sharif.edu

# Why do we need another layer of defense?

| | |
|---|---|
| **Starting point** | NX, canaries, and ASLR leave some attacks possible |
| **Problem** | A corrupted callback can still reach existing code |
| **CFI** | Which destinations may this callsite use? |
| **PAC** | Has this protected pointer been altered? |
| **MTE** | Does this memory access have a matching tag? |
| **Checkpoint** | Apply each check to the same heap callback |








# A stale callback can redirect execution

## Running example

```
struct Item {
  char name[16];
  void (*done)(struct Item *);
};
free(p);
/* attacker reuses the region */
p->done(p);
```

1 free(p) leaves a dangling pointer.

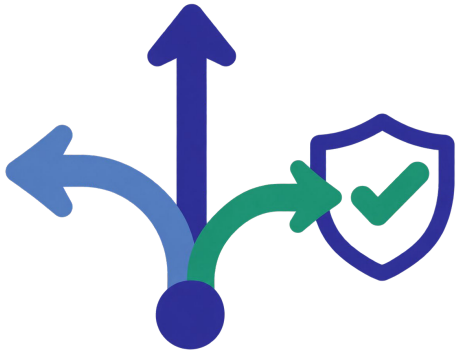
2 Attacker-controlled data replaces done.

3 The stale call uses the replacement target.

Assume control of the reused bytes and a known executable target. Which check could stop this?

# How can a corrupted pointer still hijack execution?

**Part 1: Control-flow integrity and protected returns**



# An attacker can redirect calls and returns



**FORWARD EDGE**

A corrupted callback changes an indirect call's destination. This is a forward edge.

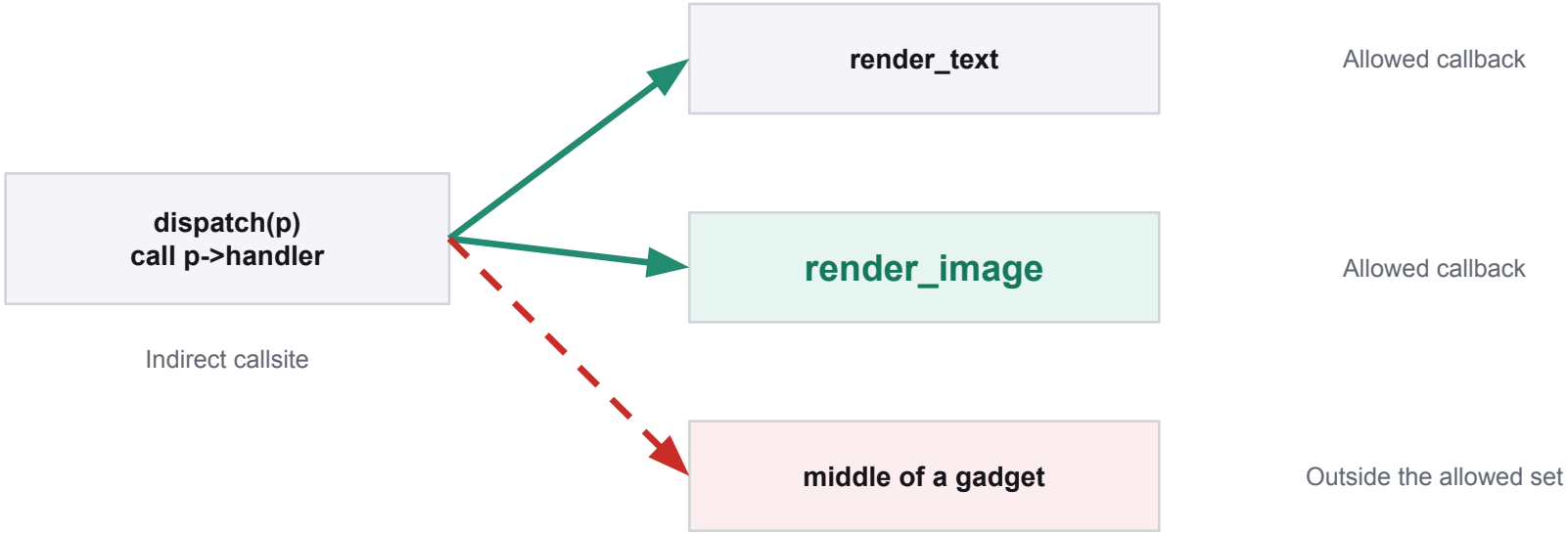
**BACKWARD EDGE**

A corrupted return address changes where a function returns. This is a backward edge.

**A forward-edge call check alone does not protect return addresses.**

# A CFI policy assigns allowed targets to a callsite

**Illustrative policy: Allowed(dispatch) = {render_text, render_image}**




**The runtime must reject a target outside this callsite's allowed set before transferring control.**

# A check must guard the target that the call uses

```
target = p->handler;
if (!allowed(site, target))
    trap();
call target;
```

| | |
|---|---|
| **Load** | load the pointer once |
| **Check** | validate this target for this site |
| **Transfer** | call the validated value |

**A separate reload from mutable memory can invalidate the check.**

# Coarse policies admit more targets

**COARSE POLICY**

**Accept any permitted function entry.**
**An attacker may select a valid**
**but unintended function.**

**MORE PRECISE POLICY**

**Restrict targets using callsite,**
**function type, or additional context.**
**Smaller sets constrain more choices.**

**An accepted control transfer can still violate an application’s authorization policy.**

Policy precision trades off analysis, compatibility, runtime cost, and residual attack choices.

# Type-based CFI still allows same-type substitutions

```
void preview(Packet *p);
void release(Packet *p);

void (*handler)(Packet *);
handler = release;
handler(p);
```

| | |
|---|---|
| **Type** | **both functions have the same type** |
| **Policy** | **a type check may accept either** |
| **Meaning** | **release may be wrong for this request** |

**A matching function type does not establish the caller’s permission to release data.**

Synthetic example of residual behavior under a type-based target policy.

# Why do returns need a shadow stack?



**THE PROBLEM**

A saved return address lives beside ordinary stack data.
An overflow may overwrite it.
ret could then follow the attacker's address.

**SHADOW STACK**

A separate protected stack records return addresses in call order.
It provides a trusted record of where functions must return.

**The shadow stack must resist writes that can corrupt the ordinary stack.**

# A shadow stack protects the expected return address



| ORDINARY STACK | PROTECTED SHADOW STACK |
| --- | --- |
| `call records return = A` | `call records return = A` |
| `ordinary return slot = gadget` | `protected top remains A` |
| `corrupted return differs from A` | `return mismatch stops the transfer` |

**The shadow record must resist the same memory-corruption primitive that changed the ordinary stack.**

# CFI deployment requires complete coverage

| | |
|---|---|
| **POLICY** | **A conservative analysis can enlarge the permitted target set** |
| **COVERAGE** | Uninstrumented code and excluded calls can leave gaps |
| **INTEGRATION** | Shared libraries, JITs, assembly, and exceptions need explicit support |
| **FAILURE** | A violation response must preserve the enforcement guarantee |

**Compiler, linker, runtime, and operating-system support determine the delivered protection.**

# Could we detect an altered pointer before using it?

**Part 2: Pointer Authentication Code and PAC contexts**



# PAC binds a pointer to a key and context

$$\text{PAC} = F_K(\text{address}, \text{context})$$

### 1 Sign

Compute a code over address and context using a secret key

**create signed pointer**

### 2 Store

**Keep address and PAC in the pointer's representation**

**PAC width varies**

### 3 Authenticate

Verify with the same key and context before use

**recover usable address**

**Changing the address usually invalidates its PAC. The protection depends on correct signing and authentication.**

# Return-address signing uses a frame context

```
entry:
  paciasp
  save x30
  /* function body */
exit: restore x30 and entry SP
  autiasp
  ret
```

| | |
|---|---|
| x30 | AArch64 link register |
| SP | same signing context at exit |
| Check | authenticate before return use |

**A changed return address should fail authentication under the original key and context.**

# Context binding constrains pointer substitution



| REUSED CONTEXT | DISTINCT CONTEXT |
| --- | --- |
| signed pointer for slot X | signed pointer for slot X |
| copy to slot Y with same context | slot Y uses a distinct context |
| authentication may succeed | authentication should reject |

**A discriminator can bind a signature to storage location, pointer type, or another policy-relevant context.**

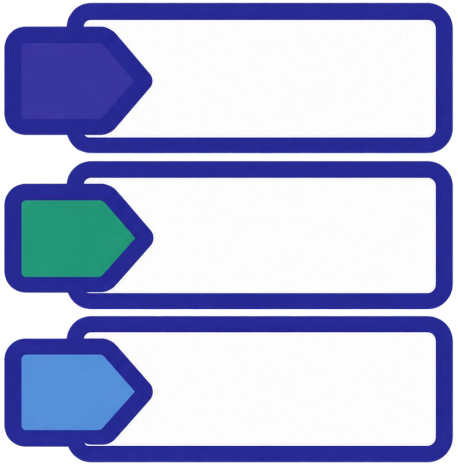
# PAC’s remaining assumptions

| | |
|---|---|
| WIDTH | **A finite PAC leaves a nonzero probability of a successful guess** |
| CONTEXT | Reusing a compatible signed pointer may preserve validity |
| SIGNING | A bug may expose an unintended signing or authentication operation |
| COVERAGE | Only pointers that the ABI signs and authenticates receive this protection |

**PAC authenticates pointer values. Separate checks must enforce object bounds, lifetime, and application policy.**

# Could we detect the stale access before the call?

**Part 3: ARM MTE (Memory Tagging Extension) and checked memory accesses**



# MTE compares logical and allocation tags

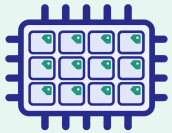
### Pointer



A 4-bit logical tag
travels with the address

**tag is metadata**

### Memory



**Each aligned 16-byte granule has a 4-bit allocation tag**

**16 possible values**

### Checked access



Hardware compares the tags
for a tagged load or store

**mismatch reports fault**

**Software must tag allocations and enable the relevant memory mappings and tag checks.**

# Retagging can detect a stale pointer



### 1 Allocate

Object memory tag = A
pointer p keeps tag A

**access matches**

### 2 Free

**Allocator changes the region to allocation tag E**

**lifetime ends**

### 3 Stale use

Old p still carries tag A
A differs from E

**tag mismatch**

**Reuse can assign a matching tag later. MTE reduces the risk of use-after-free without proving lifetime safety.**

# Worked MTE example: a 20-byte allocation



**Assume aligned p, logical tag A, and a 20-byte C allocation.**

| Memory region | Offsets from p | Allocation tag |
|---|---|---|
| Object granule 1 | 0–15 | A |
| Object granule 2 | 16–31 | A |
| Neighbor granule | 32–47 | E |

**p[19]: in bounds and tag match**
**p[25]: out of bounds but tag match**
**p[32]: tag mismatch with neighboring granule E**

**Only offsets 0–19 belong to the object. Two A-tagged granules cover 32 bytes.**

# Granule matching is coarser than object bounds

```c
char *p = malloc(20);
/* tagged extent: 32 bytes */
p[19] = 1;  // in bounds
p[25] = 1;  // out of bounds
/* both access tag A */
```

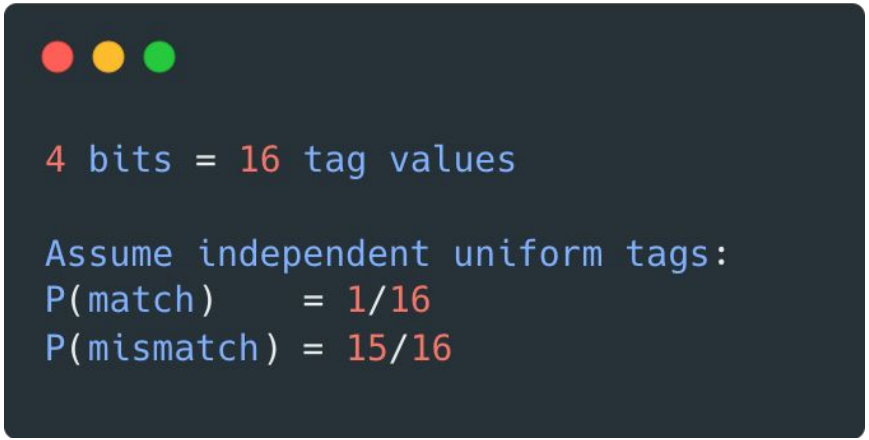
**Bounds** — valid offsets are 0 through 19

**Tags** — offset 25 remains in granule A

**Result** — MTE may miss this invalid write

**A matching tag does not prove that the access lies within the C object.**

# Tag collisions make detection probabilistic




**Model** — independent uniform tag choices

**Access** — one comparison of unrelated tags

**Caveat** — allocator policies change the odds

**15/16 is a model for one comparison, not a universal exploit-blocking rate.**

# Synchronous and asynchronous MTE faults

**SYNCHRONOUS**

**Fault at the offending access.**
**The checked memory operation does not complete.**

**ASYNCHRONOUS**

**Record the mismatch and report later.**
**Additional execution and memory operations may occur first.**

**Delayed reporting provides weaker immediate containment than stopping at the faulting access.**

# MTE coverage depends on software policy

| | |
|---|---|
| **HEAP** | **Allocator tagging and retagging define object regions** |
| **STACK** | Compiler support must assign and maintain stack-object tags |
| **LIMITS** | Same-tag granules and intra-object writes can escape detection |
| **TYPE** | A matching tag does not establish the object’s dynamic type |

**MTE tags are not secrets or permissions. Memory-safe design and testing still address uncovered errors.**

# The same attack has several enforcement points

## Return to the stale callback: where does each defense check?

| | |
|---|---|
| NX / ASLR | Execution permissions and uncertain addresses |
| CFI | The permitted destination of an indirect transfer |
| PAC | The authenticity of a protected pointer value |
| MTE | The tag relation at a checked memory access |

Root-cause repairs and memory-safe code remain necessary alongside these mitigations.

# Comparison of the enforcement guarantees

| Defense | Check / protected state | Important limit |
|---|---|---|
| **Forward CFI** | Indirect target satisfies policy | Accepted targets may still be wrong |
| **Shadow stack** | Protected return history | Does not protect heap callbacks |
| **PAC** | Pointer matches key and context | Finite code, reuse, signing coverage |
| **MTE** | Tags match at a checked access | Granule gaps, collisions, fault mode |

**Coverage depends on the deployed scheme. No row alone establishes complete memory safety.**

# Checkpoint: revisit the stale callback

**Assume a dangling heap pointer**

```
struct Item {
  char name[16];
  void (*done)(struct Item *);
};
/* p survives free and reuse */
p->done(p);
```

1 At which memory access could MTE report a fault?

2 Could a same-type replacement pass CFI?

3 Would a copied PAC remain valid in this context?

**MTE checks the access. CFI checks the target policy. PAC checks the signed pointer and context.**

# Next: vulnerability discovery and program analysis



## Session 8 examines how to find and diagnose remaining bugs.

| | |
|---|---|
| **Fuzzing** | **Exercise inputs and collect failures** |
| **Static analysis** | **Reason about code without running each path** |
| **Taint analysis** | **Track data from untrusted sources** |
| **Symbolic execution** | **Explore path conditions with constraints** |

**Today's review: locate the check, state its scope, and identify a concrete failure assumption.**

# References



| | |
|---|---|
| **COURSES** | Berkeley CS 161, Ch. 4. Stanford CS 155, Lecture 3. Prior-term control-hijacking defenses and ROP supplement. |
| **CFI PAPER** | Abadi, Budiu, Erlingsson, and Ligatti. Control-Flow Integrity: Principles, Implementations, and Applications, CCS, 2005. |
| **MTE PAPER** | Kostya Serebryany. ARM Memory Tagging Extension and How It Improves C/C++ Memory Safety, USENIX ;login:, 2019. |
| **DOCUMENTS** | LLVM: CFI, Pointer Authentication, ShadowCallStack. Arm MTE white paper. Linux arm64 PAC and MTE documentation. |